

제3회 한국천문연구원-카이스트
천문우주 AI 경진대회
본선 설명회

“미션: 지구 우주환경을 변화시키는 태양풍의 속도를 예측하라”

본선 - 스코어링 기준 및 환경

스코어링 기준을 알고 해야 좋은 성적을 거둘 수 있다. 본선의 환경은?

AI 개발 환경

- 웹 환경의 대회 운영 플랫폼에서 진행(별도 프로그램 설치 불필요)
- 팀별 1대의 전용 A100 GPU VM 인스턴스 제공
- 문제에 최적화된 AI Model 개발환경 사전 설정
- 예시로 주어지는 성능이 낮은 프로그램의 성능을 최대한 높이는 과정

문항

과거 120 h



태양 관측 영상

20 × 2채널

6시간 간격 · 193Å / 211Å



태양풍 속도

20개 관측값

과거 5일 6시간 간격



예측값

12개

미래 6-72시간 · 6시간 간격

- 기본 성능을 내는 Model 예제를 제공 → 성능 개선
- 학습 후 결과를 제출 → 리더보드 성능 순위 공개
- private dataset 검증 및 대회 종료 후 재현 검증

스코어링 기준 - Public/Private Data

- 전체 데이터셋 중 학습 데이터 67%, 검증 데이터 8% 평가 데이터 25%로 구성
- 평가 데이터는 Public 데이터 12.5%, Private(Hidden) 12.5% 로 구성됨
- 대회 기간 중 실시간 순위는 Public 데이터 기준으로 공개
- 대회 종료 후 최종 순위는 Private 데이터와 Public 데이터를 합산하여 산출

학습데이터
(67%)

검증
데이터
(8%)

평가 데이터
Public
(12.5%)

평가 데이터
Private
(12.5%)

데이터셋 디렉토리 구조

디렉토리 구조

dataset/

```

├─ dataset_info.json
├─ train/                      9,607 samples
│   ├── inputs.csv
│   ├── targets.csv
│   ├── 193/*.png
│   └── 211/*.png
├─ validation/                1,199 samples
│   ├── inputs.csv / targets.csv
│   └── 193/*.png / 211/*.png
└─ test/                      3,868 samples
    ├── inputs.csv
    ├── test_ids.csv
    └── 193/*.png / 211/*.png
  
```

CSV 스키마

inputs.csv

41 COLS

sample_id | image_00 ... image_19 | wind_00 ... wind_19
train / validation / test 공통 · wind 단위 km/s

targets.csv

13 COLS

sample_id | target_00 | target_01 ... target_11
train / validation에만 제공 · 6-72시간, 6시간 간격

test_ids.csv

1 COL

sample_id
test inputs의 ID를 통해 익명화 된 이미지 쌍 inference

submission_example.csv

13 COLS

sample_id | target_00 | target_01 ... target_11
test sample 3,868행 · 예측값은 km/s · 컬럼 순서 유지

image_XX의 파일명을 193와 211에서 각각 읽어 동일 시점의 2채널 입력으로 구성

운영 규정 - 외부 데이터 / Data Leakage

- 아래와 같은 행위들은 부정행위로 간주되어 불이익을 받을 수 있음
 - 대회에서 제공하는 데이터 이외의 데이터를 사용하는 경우
 - 예측 결과를 제출할 때 모델을 사용하지 않고 사람이 직접 처리한 결과를 제출하는 경우
 - 제출한 코드가 주어진 환경에서 정상적으로 동작하지 않는 경우
 - 제출한 코드로 예측한 결과가 제출한 결과와 성능이 크게 차이나는 경우
 - Pretrained Model 의 가중치를 활용하는 경우 (구조를 가져와서 weight initialization 후 처음 부터 학습하는 경우는 허용)
 - Test 데이터의 Target 정보를 학습에 사용 하여 Data Leakage 가 발생하는 모델링을 한 경우

베이스라인 예제 코드 개요

- 1. DataLoader
- 2. Model Architecture
- 3. Train
- 4. Evaluation
- 5. Submission

6-hour Solar Wind Speed Prediction — Baseline

이 Baseline 노트북 하나로 데이터 로드, train/validation 분리, GPU 학습, validation 평가, test 추론과 최종 제출 분인 `submission.csv` 생성을 수행합니다.

- 입력: 과거 5일의 193 Å/211 Å 영상 20쌍과 태양풍 속도 20개(6시간 간격)
- 출력: 마지막 입력 이후 6~72시간 태양풍 속도 12개(6시간 간격)

```
In [9]: from pathlib import Path
import gc
import hashlib
import json
import math
import os
import random
import time

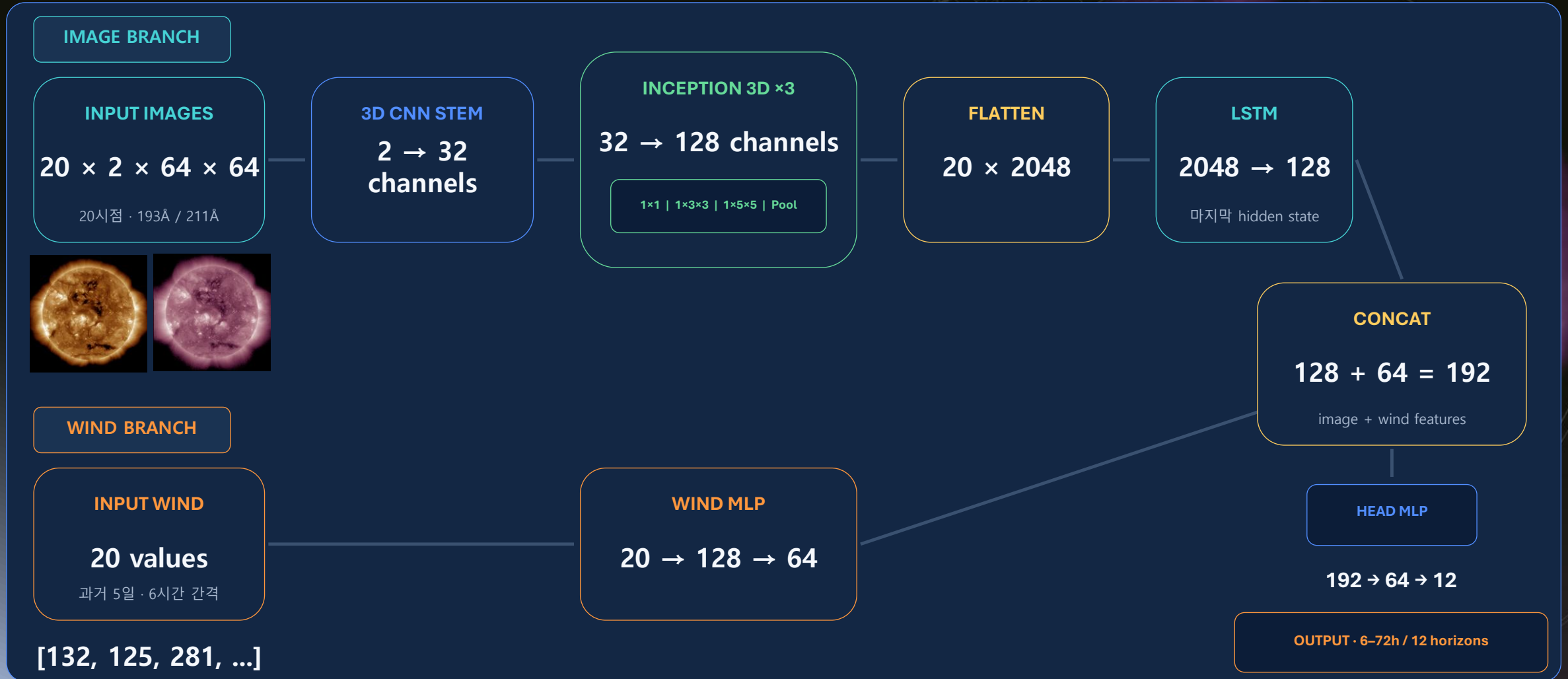
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from PIL import Image
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils.data import DataLoader, Dataset

# Python, NumPy, PyTorch의 seed를 같게 설정
SEED = 777
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# notebook과 같은 위치의 기본 데이터/출력 경로를 사용합니다.
DATA_ROOT = Path(os.getenv("SM_DATA_ROOT", "competition_dataset_6h"))
OUTPUT_DIR = Path(os.getenv("SM_OUTPUT_DIR", "outputs/baseline_6h"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

IMAGE_SIZE = 64
CHANNELS = ("193", "211")
RMSE_EPSILON = 1e-8
BATCH_SIZE = int(os.getenv("SM_BATCH_SIZE", "256"))
EPOCHS = int(os.getenv("SM_EPOCHS", "50"))
NUM_WORKERS = int(os.getenv("SM_NUM_WORKERS", "4"))
# CUDA를 사용할 수 있으면 GPU의 mixed precision(AMP)을 자동으로 사용합니다.
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
USE_AMP = DEVICE.type == "cuda"
PIN_MEMORY = DEVICE.type == "cuda"
if hasattr(torch, "set_float32_matmul_precision"):
    torch.set_float32_matmul_precision("high")
if DEVICE.type == "cuda":
    torch.backends.cudnn.benchmark = True
else:
    print("WARNING: CUDA Unavailable")
```


베이스라인 예제 코드 구조



베이스라인 - 기초적인 성능 향상 가이드

BASELINE CODE

baseline.ipynb

```

01 IMAGE_SIZE = 64
02 CHANNELS = ("193", "211")
03
04 resampling = Image.Resampling.BILINEAR
05 image.convert("L").resize(
06     (IMAGE_SIZE, IMAGE_SIZE), resampling
07 )
08
09 images = np.asarray(
10     self.image_array[self.image_indexes[row_index]],
11     dtype=np.float32
12 ) / 255.0
13
14 self.wind = inputs[WIND_COLUMNS] \
15     .to_numpy(np.float32) / 1000.0

```

WHAT TO TRY

예시값 ≠ 최적값

이미지 해상도 64 → 96 / 128 / 256 등 적용 가능

세부 구조 보존 가능성 ↔ GPU 메모리·학습 시간 증가

정규화 채널별 mean / std

통계는 train split에서만 계산하고 validation/test에는 재사용

표현 방식 Scaling · clipping · cropping

약한 구조 강조 가능성 및 유효 정보 손실을 함께 확인

검증 변환된 sample을 직접 시각화

동일 seed·동일 학습 budget으로 한 요소씩 비교

충분히 데이터들에 대해 EDA를 수행하고 얻은 인사이트를 적용하기를 적극 권장

베이스라인 - 기초적인 성능 향상 가이드

BASELINE CODE

baseline.ipynb

```

01 BATCH_SIZE = int(os.getenv(
02     "SW_BATCH_SIZE", "256"))
03 EPOCHS = int(os.getenv("SW_EPOCHS", "20"))
04
05 optimizer = torch.optim.Adam(
06     model.parameters(), lr=1e-4)
07 scheduler = ReduceLROnPlateau(
08     optimizer, mode="min", factor=0.25,
09     patience=1, min_lr=1e-6)
10 patience = 5
11
12 val_rmse = run_epoch(val_loader, training=False)
13 scheduler.step(val_rmse)
14 if val_rmse < best_val_rmse:
15     torch.save(..., checkpoint_path)
16 elif epochs_without_improvement >= patience:
17     break

```

HOW TO TUNE

예시값 ≠ 최적값

Epoch

전체 데이터를 몇 번 학습할 것 인가?

Epoch 이 너무 크면 train distributio에 overfitting
너무 작으면 underfitting이 될 위험이 존재함

Batch

한 번에 몇 개의 입력을 학습할 것인가?

batch 변경 시 optimizer 호출 수에 차이 발생
learning rate와 학습 시간도 재검토

Scheduler

Learning Rate 를 가변적으로 적용 가능

급격한 감소는 남은 개선 가능성을 제한할 수 있음
너무 큰 learning rate 은 수렴을 방해할 수 있음

Patience

Scheduler, Epoch 개선이 없으면 멈춤

Patience가 너무 큰 경우 수렴이후에도 overfitting 가능
Patience가 너무 작은 경우 수렴되기 전 underfitting 가능

다른 Optimizer · Weight Decay · Gradient Clipping 등 다양한 방식 시도 가능

베이스라인 - 평가지표 활용 가이드

지표	무엇을 측정?	특징	활용
RMSE	오차 제곱의 평균 후 제곱근	큰 오차에 더 큰 패널티	경진대회 메인 평가 지표
MAE	절대 오차의 평균	직관적이고 이상치 영향이 상대적으로 작음	전반적 오차 규모 진단
Pearson Correlation (r)	예측-실제의 선형 동조성	스케일·편향 오차를 직접 측정하지 않음	패턴 추종 여부 보조 진단
Persistence	마지막 값이 계속 유지될 경우의 RMSE	모델링이 필요하지 않음	객관적 모델 성능 진단

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

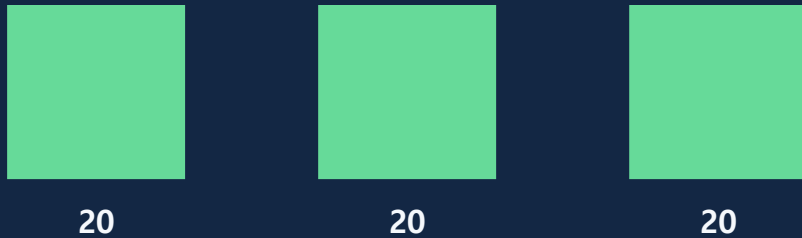
$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$r = \frac{\sum_i (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_i (y_i - \bar{y})^2} \sqrt{\sum_i (\hat{y}_i - \bar{\hat{y}})^2}}$$

평가지표 - RMSE

해당 대회에의 평가지표인 RMSE는 얼마나 큰 오차를 잘 억제하는지가 중요!

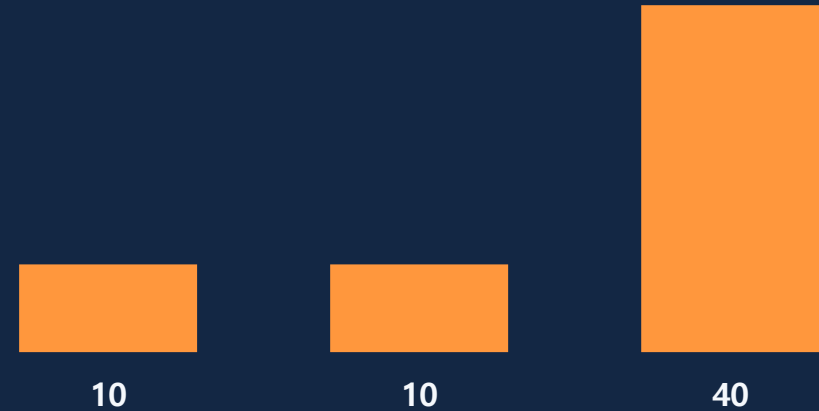
MODEL A



MAE = 20.0

RMSE = 20.0

MODEL B



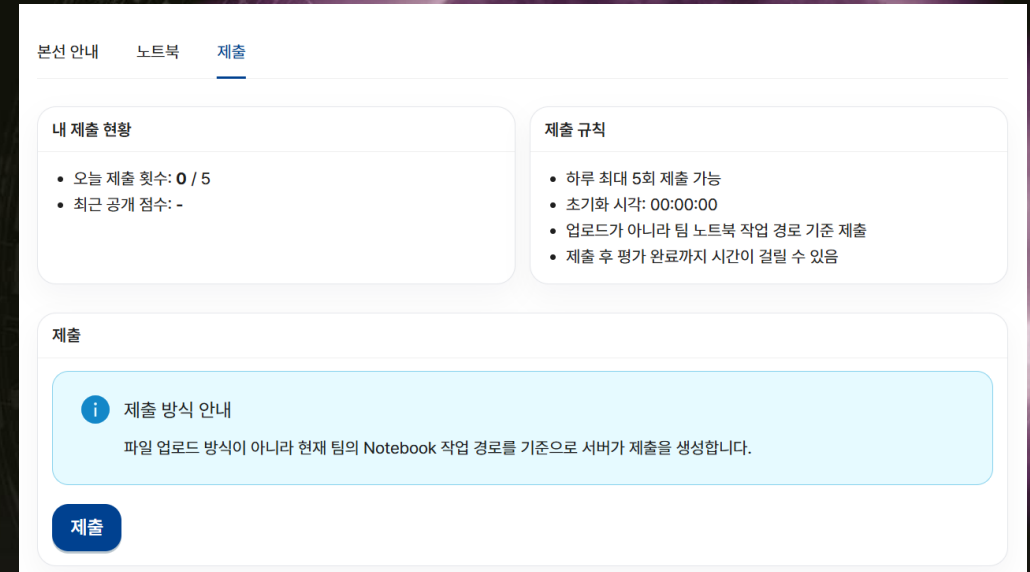
MAE = 20.0

RMSE = 24.5

같은 MAE 20.0 ————— 큰 단일 오차 때문에 RMSE 증가

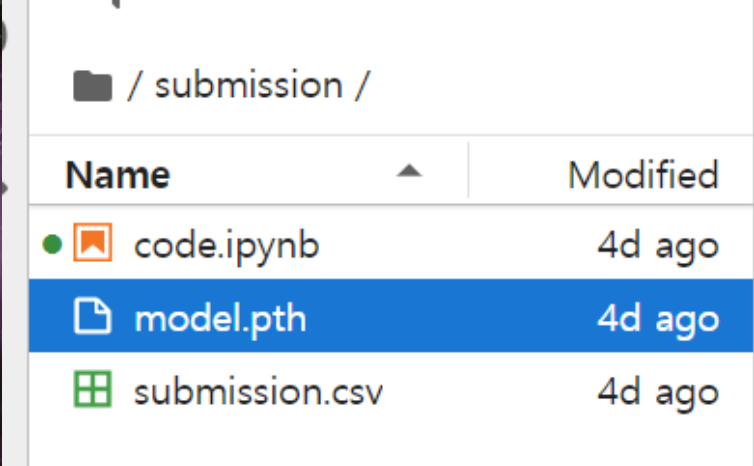
제출 방법

- 본선 탭에 존재하는 플랫폼 내의 제출 버튼을 클릭하여 제출
- 주피터 노트북 환경은 경진대회>본선>노트북 탭에서 이동 가능



제출되는 항목

- 구현에 사용한 코드
 - 이름은 반드시 code.ipynb
 - 이 파일 외의 코드는 제출되지 않음
- 예측 결과를 저장한 submission.csv 파일
- 추론에 사용한 model.pth 파일
- 세 파일 외의 다른 파일은 제출되지 않음
- 세 파일을 submission/ 폴더 아래에 저장



/ submission /	
Name	Modified
code.ipynb	4d ago
model.pth	4d ago
submission.csv	4d ago

코드 작성 시 유의사항

- code.ipynb 파일 하나만으로 동작하도록 구성
 - 다른 폴더에서 import 하는 방식은 지양
- 필요한 주요 python 패키지는 기본적으로 제공하나, 별도의 패키지가 필요한 경우 code.ipynb 파일 최상단에 !pip install 으로 명시
- 제공되는 sample_code.ipynb 파일은 읽기 전용으로 제공되니 해당 파일을 Duplicate 하신 후 code.ipynb로 파일명을 코드를 작성
- 동일 계정으로 여러 기기에서 동시에 접속할 수 없으며, 다른 기기에서 로그인하면 기존 접속 세션이 만료

Q&A

